DSA – Sec A

Lab – 08

Instructor	**Mr. Shehryar**
Lab Instructors	• Muhammad Taha • Rimsha Aamir • Adeel Zafar

Department of Computer Science, ITU, Lahore.

1. **Implement Queue using Linked List(enqueue, dequeue, rare, front)**

   Follow the below steps to solve the problem:

   - Create a class QNode with data members integer data and QNode* next
     - A parameterized constructor that takes an integer x value as a parameter and sets data equal to x and next as NULL
   - Create a class Queue with data members QNode front and rear
   - Enqueue Operation with parameter x:
     - Initialize QNode* temp with data = x
     - If the rear is set to NULL then set the front and rear to temp and return(Base Case)
     - Else set rear next to temp and then move rear to temp
   - Dequeue Operation:
     - If the front is set to NULL return(Base Case)
     - Initialize QNode temp with front and set front to its next
     - If the front is equal to NULL then set the rear to NULL
     - Delete temp from the memory

2. **Flattening a Linked List**

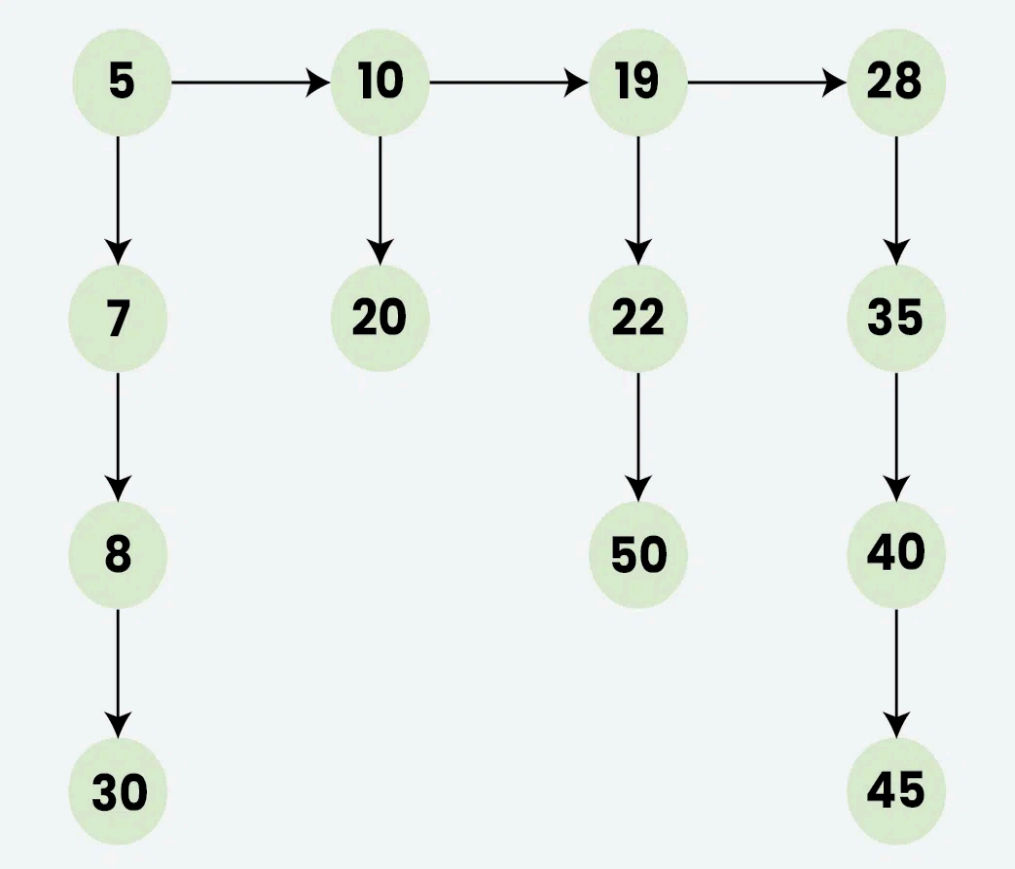
   Given a Linked List of size N, where every node represents a sub-linked-list and contains two pointers:

   - **next** pointer to the next node
   - **bottom** pointer to a linked list where this node is head.

   Each of the sub-linked-list is in sorted order. Flatten the Link List such that all the nodes appear in a single level while maintaining the sorted order.

   Note: The flattened list will be printed using the bottom pointer instead of the next pointer.

Example:



Output: 5->7->8->10->19->22->28->30->50

For testing you can use hard coded Linked list like:

```
Node* head = new Node(5);
head->bottom = new Node(7);
head->bottom->bottom = new Node(8);
head->bottom->bottom->bottom = new Node(30);

head->next = new Node(10);
head->next->bottom = new Node(20);

head->next->next = new Node(19);
head->next->next->bottom = new Node(22);
head->next->next->bottom->bottom = new Node(50);

head->next->next->next = new Node(28);
```

3. **Find Pairs with a Given Sum/Target in a Sorted Doubly Linked List in O(N)**
   Given a sorted doubly linked list of positive distinct elements, the task is to find pairs in a doubly-linked list whose sum is equal to given value target.

   Example:

   - Input: 1 <-> 2 <-> 4 <-> 5 <-> 6 <-> 8 <-> 9, target sum 7
   - Output: (1, 6), (2, 5)

   You don't need to read input or print anything. Your task is to complete the function findPairsWithGivenSum() which takes head node of the doubly linked list and an integer target as input parameter and returns an array of pairs. If there is no such pair return empty array.

   Expected Time Complexity: O(N)

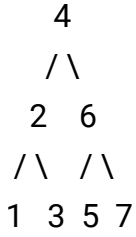
4. **Convert a doubly linked list to BST**

   Problem Statement:
   Write a function to convert a sorted doubly linked list into a balanced Binary Search Tree (BST). The tree should have minimal height, and the in-order traversal of the BST should produce the same sequence as the doubly linked list.

   Sample Input:

   Doubly Linked List: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <-> 7

   Sample Output:



5. Design Browser History using doubly linked list.

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

- BrowserHistory(string homepage) Initializes the object with the homepage of the browser.
- void visit(string url) Visits url from the current page. It clears up all the forward history.
- string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.
- string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Example:

Input:

["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]

[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]

Output:

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

Explanation:

BrowserHistory browserHistory = new BrowserHistory("leetcode.com");

browserHistory.visit("google.com"); // You are in "leetcode.com". Visit "google.com"

browserHistory.visit("facebook.com"); // You are in "google.com". Visit "facebook.com"

browserHistory.visit("youtube.com"); // You are in "facebook.com". Visit "youtube.com"

browserHistory.back(1); // You are in "youtube.com", move back to "facebook.com" return "facebook.com"

browserHistory.back(1); // You are in "facebook.com", move back to "google.com" return "google.com"
browserHistory.forward(1); // You are in "google.com", move forward to "facebook.com" return "facebook.com"
browserHistory.visit("linkedin.com"); // You are in "facebook.com". Visit "linkedin.com"
browserHistory.forward(2); // You are in "linkedin.com", you cannot move forward any steps.
browserHistory.back(2); // You are in "linkedin.com", move back two steps to "facebook.com" then to "google.com". return "google.com"
browserHistory.back(7); // You are in "google.com", you can move back only one step to "leetcode.com". return "leetcode.com"

6. Design Circular Deque

- MyCircularDeque(int k) Initializes the deque with a maximum size of k.
- boolean insertFront() Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.
- boolean insertLast() Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.
- boolean deleteFront() Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.
- boolean deleteLast() Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.
- int getFront() Returns the front item from the Deque. Returns -1 if the deque is empty.
- int getRear() Returns the last item from Deque. Returns -1 if the deque is empty.
- boolean isEmpty() Returns true if the deque is empty, or false otherwise.
- boolean isFull() Returns true if the deque is full, or false otherwise.

Example:
Input:
["MyCircularDeque", "insertLast", "insertLast", "insertFront", "insertFront", "getRear", "isFull", "deleteLast", "insertFront", "getFront"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]

Output:
[null, true, true, true, false, 2, true, true, true, 4]

Explanation

```
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
myCircularDeque.insertLast(1);  // return True
myCircularDeque.insertLast(2);  // return True
myCircularDeque.insertFront(3); // return True
myCircularDeque.insertFront(4); // return False, the queue is full.
myCircularDeque.getRear();      // return 2
myCircularDeque.isFull();       // return True
myCircularDeque.deleteLast();   // return True
myCircularDeque.insertFront(4); // return True
myCircularDeque.getFront();     // return 4
```